

```

`timescale 1ns / 1ps
module top (
    input wire i_clk,
    input wire i_rst,
    output reg o_current_val,
    output wire o_pattern_valid,
    output wire o_clk_slow
);

    wire enable;
    wire data;

    clockdiv cdiv (
        .i_clk(i_clk),
        .i_rst(i_rst),
        .o_en(enable),
        .o_clk(o_clk_slow)
    );

    detector uut_det (
        .i_clk(i_clk),
        .i_en(enable),
        .i_dat_in(data),
        .i_rst(i_rst),
        .o_pattern_valid(o_pattern_valid)
    );

    generator uut_gen (
        .i_clk(i_clk),
        .i_en(enable),
        .i_rst(i_rst),
        .o_pattern_dat(data)
    );

    always @(posedge i_clk or posedge i_rst) begin
        if(i_rst) begin
            o_current_val <= 1'b0;
        end
        else if(enable) begin
            o_current_val <= data;
        end
    end

endmodule

module clockdiv #(
    parameter DESIRED_FREQ = 1,
    parameter IP_FREQ = 125000000
) (
    input wire i_clk,
    input wire i_rst,
    output reg o_en,
    output reg o_clk
);

    localparam integer SETPOINT = (IP_FREQ) / (2 * DESIRED_FREQ);
    localparam integer COUNTER_WIDTH = $clog2(SETPOINT);

    reg [COUNTER_WIDTH-1:0] counter = 'b0;

    always @(posedge i_clk) begin
        if (i_rst) begin
            counter <= 'b0;
            o_en <= 1'b0;
            o_clk <= 0;
        end else if (counter == SETPOINT - 1) begin
            counter <= 'b0;
            o_clk <= ~o_clk;
            o_en <= ~o_clk;
        end
    end
endmodule

```

```

        end else begin
            counter <= counter + 1'b1;
            o_en <= 1'b0;
        end
    end
endmodule

module generator (
    input wire i_clk,
    input wire i_rst,
    input wire i_en,
    output wire o_pattern_dat
);

    localparam [119:0] TEST_STREAM = {
        4'b0010, // Direct 0010
        4'b0100, // Direct 0100
        7'b0010010, // Overlapping 0010
        7'b0100100, // Overlapping 0100
        5'b00100, // Partial overlap
        5'b01001, // Partial overlap
        8'b00000000, // Continuous 0
        8'b11111111, // Continuous 1
        8'b00110010, // Near miss + 0010
        8'b01010010, // Near miss + 0010
        8'b11001001, // 0010 in arbitrary data
        8'b10100100, // 0100 in arbitrary data
        8'b00010000, // Long zero / partial
        8'b11101111, // Long one / partial
        12'b001001000100, // Multiple patterns
        12'b010000100010 // Multiple patterns
    };

    reg [6:0] bit_count;

    assign o_pattern_dat = TEST_STREAM[119-bit_count];

    always @(posedge i_clk or posedge i_rst) begin
        if (i_rst) begin
            bit_count <= 7'd0;
        end else if (i_en) begin
            if (bit_count == 7'd119) bit_count <= 7'd0;
            else bit_count <= bit_count + 1'b1;
        end
    end
endmodule

module detector (
    input wire i_clk,
    input wire i_en,
    input wire i_dat_in,
    input wire i_rst,
    output reg o_pattern_valid
);

    reg [2:0] state;

    localparam RESET = 3'b000;
    localparam INIT = 3'b001;
    localparam Z0 = 3'b010;
    localparam ZZ = 3'b011;
    localparam ZZ0 = 3'b100;
    localparam ZOZ = 3'b101;

    always @(posedge i_clk or posedge i_rst) begin
        if (i_rst) begin
            state <= RESET;
            o_pattern_valid <= 1'b0;
        end
    end
endmodule

```

```
end else if (i_en) begin
  case (state)
    RESET: begin
      state <= i_dat_in ? RESET : INIT;
      o_pattern_valid <= 0;
    end
    INIT: begin
      state <= i_dat_in ? Z0 : ZZ;
      o_pattern_valid <= 0;
    end
    Z0: begin
      state <= i_dat_in ? RESET : Z0Z;
      o_pattern_valid <= 0;
    end
    ZZ: begin
      state <= i_dat_in ? ZZ0 : ZZ;
      o_pattern_valid <= 0;
    end
    ZZ0: begin
      // state <= i_dat_in ? RESET : Z0Z; overlapping detector
      state <= i_dat_in ? RESET : RESET;
      o_pattern_valid <= i_dat_in ? 1'b0 : 1'b1;
    end
    Z0Z: begin
      // state <= i_dat_in ? Z0 : Z0Z; overlapping detector
      // state cahnge
      state <= i_dat_in ? Z0 : RESET;
      o_pattern_valid <= i_dat_in ? 1'b0 : 1'b1;
    end
    default: begin
      state <= RESET;
      o_pattern_valid <= 0;
    end
  endcase
end
end
endmodule
```